Documentation - Holistic Data Preparer

Project Summary

This project implements a complete data preprocessing pipeline for a customer credit risk dataset. The workflow includes exploratory data analysis (EDA), missing value treatment using Mean, Median, Mode, and KNN Imputation, outlier detection and treatment, feature engineering, encoding, transformations, feature scaling, clustering, and dataset export.

Financial institutions use credit risk analysis to determine whether a customer is likely to repay a loan. Raw datasets often contain missing values, outliers, inconsistent formats, and categorical variables that require preprocessing before machine learning models can be developed.

This project focuses on preparing a customer credit risk dataset using advanced preprocessing techniques to improve data quality and model performance.

Dataset Overview

Dataset contains customer demographics, financial attributes, loan information, credit score information, transaction details, and customer onboarding dates.

Feature	Description
customer_id	Unique customer identifier
age	Customer age
gender	Gender of customer
region	Customer region
education_level	Educational qualification
employment_type	Employment status
annual_income	Annual income
loan_amount	Loan amount requested

Feature	Description
credit_score	Customer credit score
transaction_count	Number of transactions
spending_ratio	Spending behavior
join_date	Customer joining date
loan_purpose	Purpose of loan
default_risk	Risk category

Objectives

- Improve data quality
  - Handle missing values
  - Detect and treat outliers
  - Engineer useful features
  - Prepare data for machine learning
- Understand the customer credit risk dataset.
  - Perform data quality assessment.
  - Handle missing values.
  - Detect and treat outliers.
  - Create meaningful features.
  - Perform categorical encoding.
  - Apply data transformation techniques.
  - Scale numerical variables.
  - Export the final cleaned dataset.

Methodology

Data Loading → EDA → Missing Value Handling → Outlier Treatment → Feature Engineering → Encoding → Transformation → Scaling → Export

Missing Value Handling

Mean, Median, Mode, and KNN Imputation were used. Mean replaces values with averages, Median is robust to outliers, Mode fills categorical values, and KNN predicts missing values from similar records.

Outlier Treatment

Outliers were detected using Z-Score and IQR methods and treated using capping and Winsorization.

Feature Engineering

Created Debt-to-Income Ratio and extracted date-based features such as Join Year, Join Month, and Join Weekday.

Encoding

Applied Ordinal Encoding and One-Hot Encoding to convert categorical variables into machine-readable form.

Transformation & Scaling

Used Log, Square Root, Reciprocal, Box-Cox, Yeo-Johnson transformations and StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler, and Normalizer.

Step 1

Code:

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

Interpretation: Performs a preprocessing or analytical operation.

Step 2

Code:

```
from sklearn.impute import SimpleImputer
from sklearn.impute import KNNImputer

from scipy.stats import zscore
from scipy.stats.mstats import winsorize
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 3

Code:

```
df = pd.read_csv("/content/customer_credit_risk_dataset.csv")
```

Interpretation: Loads the dataset into a Pandas DataFrame.

Step 4

Code:

```
df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 5

Code:

```
df.tail()
```

Interpretation: Displays the last records for validation.

Step 6

Code:

```
df.shape
```

Interpretation: Shows the number of rows and columns.

Step 7

Code:

```
df.info()
```

Interpretation: Displays data types and non-null counts.

Step 8

Code:

```
df.duplicated().sum()
```

Interpretation: Checks duplicate records.

Step 9

Code:

```
df.isnull().sum()
```

Interpretation: Identifies missing values.

Step 10

Code:

```
df.isnull().sum().sum()
```

Interpretation: Identifies missing values.

Step 11

Code:

```
(df.isnull().sum() / len(df)) * 100
```

Interpretation: Identifies missing values.

Step 12

Code:

```
null_cols = df.isnull().sum()

null_cols[null_cols > 0]
```

Interpretation: Identifies missing values.

Step 13

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

sns.heatmap(
   df.isnull(),
   cbar=False,
   cmap='viridis'
)

plt.title("Missing Values Heatmap")

plt.show()
```

Interpretation: Identifies missing values.

Step 14

Code:

```
import seaborn as sns

sns.histplot(df['age'], kde=True)

plt.title("Age Distribution")

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 15

Code:

```
sns.countplot(x='gender', data=df)

plt.title("Gender Distribution")

plt.show()
```

Interpretation: Displays category frequencies.

Step 16

Code:

```
df.describe()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 17

Code:

```
from sklearn.impute import SimpleImputer

mean_imp = SimpleImputer(strategy='mean')

df[['age']] = mean_imp.fit_transform(df[['age']])
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 18

Code:

```
median_imp = SimpleImputer(strategy='median')

df[['annual_income']] = median_imp.fit_transform(
   df[['annual_income']]
)
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 19

Code:

```
mode_imp = SimpleImputer(strategy='most_frequent')

df[['gender']] = mode_imp.fit_transform(
   df[['gender']]
)
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 20

Code:

```
from sklearn.impute import KNNImputer

knn = KNNImputer(n_neighbors=5)

df[['credit_score']] = knn.fit_transform(
   df[['credit_score']]
)
```

Interpretation: KNN Imputer estimates missing values using similar observations.

Step 21

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

sns.heatmap(
   df.isnull(),
   cbar=False,
   cmap='viridis'
)
```

```
plt.title("Missing Values Heatmap")

plt.show()
```

Interpretation: Identifies missing values.

Step 22

Code:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(
   strategy='most_frequent'
)

df[['employment_type']] = imputer.fit_transform(
   df[['employment_type']]
)
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 23

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

sns.heatmap(
   df.isnull(),
   cbar=False,
   cmap='viridis'
)

plt.title("Missing Values Heatmap")

plt.show()
```

Interpretation: Identifies missing values.

Step 24

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10,8))

sns.heatmap(
   df.select_dtypes(include=['int64','float64']).corr(),
   annot=True,
   cmap='coolwarm',
   fmt='.2f'
)

plt.title("Correlation Heatmap")

plt.show()
```

Interpretation: Creates a heatmap for relationships or missing value patterns.

Step 25

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

numerical_cols = [
   'age',
   'annual_income',
   'loan_amount',
   'credit_score',
   'transaction_count',
   'spending_ratio'
]

for col in numerical_cols:

   plt.figure(figsize=(8,4))

   sns.boxplot(x=df[col])

   plt.title(f'Box Plot of {col}')

   plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 26

Code:

```
from sklearn.impute import SimpleImputer

median_imp = SimpleImputer(strategy='median')

df[['annual_income','credit_score']] = median_imp.fit_transform(
   df[['annual_income','credit_score']]
)
```

Interpretation: Mean imputation replaces missing numerical values using the average.

Step 27

Code:

```
df['loan_amount'] = df['loan_amount'].fillna(
   df['loan_amount'].median()
)
```

Interpretation: Performs a preprocessing or analytical operation.

Step 28

Code:

```
from scipy.stats import zscore
import numpy as np

z_scores = np.abs(
   zscore(df[['annual_income','loan_amount','credit_score']])
)

outliers = (z_scores > 3)

print(outliers.sum())
```

Interpretation: Detects outliers using standard deviations from the mean.

Step 29

Code:

```
for col in ['annual_income','loan_amount','credit_score']:

   Q1 = df[col].quantile(0.25)
   Q3 = df[col].quantile(0.75)
```

```
  IQR = Q3 - Q1

  lower = Q1 - 1.5 * IQR
  upper = Q3 + 1.5 * IQR

  outliers = df[
    (df[col] < lower) |
    (df[col] > upper)
  ]

  print(col, len(outliers))
```

Interpretation: Performs a preprocessing or analytical operation.

Step 30

Code:

```
for col in ['annual_income','loan_amount','credit_score']:

  lower = df[col].quantile(0.01)

  upper = df[col].quantile(0.99)

  df[col] = np.where(
    df[col] < lower,
    lower,
    df[col]
  )

  df[col] = np.where(
    df[col] > upper,
    upper,
    df[col]
  )
```

Interpretation: Performs a preprocessing or analytical operation.

Step 31

Code:

```
from scipy.stats.mstats import winsorize

df['annual_income'] = winsorize(
  df['annual_income'],
  limits=[0.01,0.01]
```

```
)

df['loan_amount'] = winsorize(
   df['loan_amount'],
   limits=[0.01,0.01]
)

df['credit_score'] = winsorize(
   df['credit_score'],
   limits=[0.01,0.01]
)
```

Interpretation: Caps extreme values to reduce outlier influence.

Step 32

Code:

```
df['debt_income_ratio'] = (
   df['loan_amount'] /
   df['annual_income']
)

df[['loan_amount','annual_income','debt_income_ratio']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 33

Code:

```
df['join_date'] = pd.to_datetime(
   df['join_date']
)
```

Interpretation: Performs a preprocessing or analytical operation.

Step 34

Code:

```
df['join_year'] = df['join_date'].dt.year
```

Interpretation: Performs a preprocessing or analytical operation.

Step 35

Code:

```
df['join_month'] = df['join_date'].dt.month
```

Interpretation: Performs a preprocessing or analytical operation.

Step 36

Code:

```
df['join_day'] = df['join_date'].dt.day
```

Interpretation: Performs a preprocessing or analytical operation.

Step 37

Code:

```
df['join_weekday'] = df['join_date'].dt.dayofweek
```

Interpretation: Performs a preprocessing or analytical operation.

Step 38

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.countplot(
   x='join_year',
   data=df
)

plt.title("Customer Join Year Distribution")
plt.xticks(rotation=45)

plt.show()
```

Interpretation: Displays category frequencies.

Step 39

Code:

```
import matplotlib.pyplot as plt

yearly_customers = df['join_year'].value_counts().sort_index()
plt.figure(figsize=(8,5))

plt.plot(
   yearly_customers.index,
```

```
    yearly_customers.values,
    marker='o'
)

plt.title("Customer Join Trend by Year")
plt.xlabel("Join Year")
plt.ylabel("Number of Customers")

plt.grid(True)

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 40

Code:

```
df['education_level'].unique()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 41

Code:

```
from sklearn.preprocessing import OrdinalEncoder

education_order = [
    [
        'Primary',
        'Secondary',
        'Graduate',
        'Post-Graduate'
    ]
]

encoder = OrdinalEncoder(
    categories=education_order
)

df[['education_level']] = encoder.fit_transform(
    df[['education_level']]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 42

Code:

```
df[['education_level']].head(10)
```

Interpretation: Displays the first records to understand dataset structure.

Step 43

Code:

```
import matplotlib.pyplot as plt

edu_counts = (
   df['education_level']
   .value_counts()
   .sort_index()
)

plt.figure(figsize=(8,5))

plt.plot(
   edu_counts.index,
   edu_counts.values,
   marker='o'
)

plt.title("Encoded Education Level Trend")
plt.xlabel("Encoded Education Level")
plt.ylabel("Count")

plt.grid(True)

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 44

Code:

```
df['gender'].unique()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 45

Code:

```
import matplotlib.pyplot as plt

gender_counts = (
   df['gender']
   .value_counts()
   .sort_index()
)

plt.figure(figsize=(6,4))

plt.bar(
   gender_counts.index,
   gender_counts.values
)

plt.title("Gender Distribution After Encoding")
plt.xlabel("Encoded Gender")
plt.ylabel("Count")

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 46

Code:

```
print(df['region'].unique())

print(df['loan_purpose'].unique())
```

Interpretation: Performs a preprocessing or analytical operation.

Step 47

Code:

```
df = pd.get_dummies(
   df,
   columns=[
      'region',
      'loan_purpose'
   ],
   dtype=int
```

```
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 48

Code:

```
region_cols = [col for col in df.columns if 'region_' in col]

df[region_cols].sum().plot(
   kind='bar',
   figsize=(8,5)
)

plt.title("Encoded Region Categories")

plt.ylabel("Count")

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 49

Code:

```
df['income_group'] = pd.cut(
   df['annual_income'],
   bins=3,
   labels=[
      'Low Income',
      'Medium Income',
      'High Income'
   ]
)

df[['annual_income','income_group']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 50

Code:

```
df['income_group'].value_counts()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 51

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.countplot(
   x='income_group',
   data=df,
   order=[
      'Low Income',
      'Medium Income',
      'High Income'
   ]
)

plt.title("Income Group Distribution")
plt.xlabel("Income Group")
plt.ylabel("Number of Customers")

plt.show()
```

Interpretation: Displays category frequencies.

Step 52

Code:

```
df['transaction_bin'] = pd.qcut(
   df['transaction_count'],
   q=4,
   labels=[
      'Q1',
      'Q2',
      'Q3',
      'Q4'
   ]
)

df[['transaction_count','transaction_bin']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 53

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))

sns.countplot(
   x='transaction_bin',
   data=df,
   order=['Q1','Q2','Q3','Q4']
)

plt.title("Transaction Count Quartiles")
plt.xlabel("Quartile")
plt.ylabel("Number of Customers")

plt.show()
```

Interpretation: Displays category frequencies.

Step 54

Code:

```
quartile_counts = (
   df['transaction_bin']
   .value_counts()
   .sort_index()
)

plt.figure(figsize=(8,5))

plt.plot(
   quartile_counts.index,
   quartile_counts.values,
   marker='o'
)

plt.title("Transaction Count Quartiles")
plt.xlabel("Quartile")
plt.ylabel("Number of Customers")

plt.grid(True)
```

```
plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 55

Code:

```
from sklearn.cluster import KMeans
kmeans = KMeans(
   n_clusters=4,
   random_state=42
)

df['kmeans_bin'] = kmeans.fit_predict(
   df[['transaction_count']]
)

df[['transaction_count','kmeans_bin']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 56

Code:

```
df['kmeans_bin'].value_counts()
```

Interpretation: Creates customer segments using clustering.

Step 57

Code:

```
cluster_means = (
   df.groupby('kmeans_bin')['transaction_count']
   .mean()
   .sort_values()
)

plt.figure(figsize=(8,5))

plt.plot(
   cluster_means.index,
   cluster_means.values,
   marker='o'
)
```

```
plt.title("Average Transaction Count by Cluster")
plt.xlabel("Cluster")
plt.ylabel("Average Transaction Count")

plt.grid(True)

plt.show()
```

Interpretation: Creates customer segments using clustering.

Step 58

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.histplot(
   df['annual_income'],
   kde=True
)

plt.title("Annual Income Before Transformation")

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 59

Code:

```
import numpy as np

df['annual_income_log'] = np.log1p(
   df['annual_income']
)

df[['annual_income','annual_income_log']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 60

Code:

```
plt.figure(figsize=(8,5))

sns.histplot(
   df['annual_income_log'],
   kde=True
)

plt.title("Annual Income After Log Transformation")

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 61

Code:

```
df['annual_income_sqrt'] = np.sqrt(
   df['annual_income']
)

df[['annual_income','annual_income_sqrt']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 62

Code:

```
plt.figure(figsize=(8,5))

sns.histplot(
   df['annual_income_sqrt'],
   kde=True
)

plt.title("Annual Income After Square Root Transformation")

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 63

Code:

```
df['annual_income_reciprocal'] = (
   1 / (df['annual_income'] + 1)
)
```

```
df[['annual_income','annual_income_reciprocal']].head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 64

Code:

```
plt.figure(figsize=(8,5))

sns.histplot(
   df['annual_income_reciprocal'],
   kde=True
)

plt.title("Annual Income After Reciprocal Transformation")

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 65

Code:

```
plt.figure(figsize=(12,8))

plt.subplot(2,2,1)
sns.histplot(df['annual_income'], kde=True)
plt.title("Original")

plt.subplot(2,2,2)
sns.histplot(df['annual_income_log'], kde=True)
plt.title("Log")

plt.subplot(2,2,3)
sns.histplot(df['annual_income_sqrt'], kde=True)
plt.title("Square Root")

plt.subplot(2,2,4)
sns.histplot(df['annual_income_reciprocal'], kde=True)
plt.title("Reciprocal")

plt.tight_layout()
plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 66

Code:

```
from scipy.stats import boxcox

df['annual_income_boxcox'], lambda_value = boxcox(
   df['annual_income']
)

print("Lambda Value:", lambda_value)
```

Interpretation: Performs a preprocessing or analytical operation.

Step 67

Code:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

sns.histplot(
   df['annual_income'],
   kde=True
)

plt.title("Before Box-Cox")

plt.subplot(1,2,2)

sns.histplot(
   df['annual_income_boxcox'],
   kde=True
)

plt.title("After Box-Cox")

plt.tight_layout()

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 68

Code:

```
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(
   method='yeo-johnson'
)

df['loan_amount_yeojohnson'] = pt.fit_transform(
   df[['loan_amount']]
)
```

Interpretation: Applies Yeo-Johnson transformation to improve normality.

Step 69

Code:

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

sns.histplot(
   df['loan_amount'],
   kde=True
)

plt.title("Before Yeo-Johnson")

plt.subplot(1,2,2)

sns.histplot(
   df['loan_amount_yeojohnson'],
   kde=True
)

plt.title("After Yeo-Johnson")

plt.tight_layout()

plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 70

Code:

```
numerical_cols = [
   'annual_income',
   'loan_amount',
   'credit_score'
]
```

Interpretation: Performs a preprocessing or analytical operation.

Step 71

Code:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[numerical_cols] = scaler.fit_transform(
   df[numerical_cols]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 72

Code:

```
print(df[numerical_cols].mean())

print(df[numerical_cols].std())
```

Interpretation: Performs a preprocessing or analytical operation.

Step 73

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.boxplot(
   data=df[['annual_income_boxcox',
         'loan_amount_yeojohnson',
```

```
        'credit_score']]
)

plt.title("Before Standard Scaling")

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 74

Code:

```
plt.figure(figsize=(8,5))

sns.boxplot(
   data=df[numerical_cols]
)

plt.title("After Standard Scaling")

plt.show()
```

Interpretation: Performs a preprocessing or analytical operation.

Step 75

Code:

```
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()

numerical_cols = [
   'annual_income',
   'loan_amount',
   'credit_score'
]

df[numerical_cols] = minmax.fit_transform(
   df[numerical_cols]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 76

Code:

```
print(df[numerical_cols].min())

print(df[numerical_cols].max())
```

Interpretation: Performs a preprocessing or analytical operation.

Step 77

Code:

```
for col in numerical_cols:

   plt.figure(figsize=(7,4))

   sns.histplot(
      df[col],
      kde=True
   )

   plt.title(f'MinMax Scaled {col}')

   plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 78

Code:

```
from sklearn.preprocessing import MaxAbsScaler

maxabs = MaxAbsScaler()

numerical_cols = [
   'annual_income',
   'loan_amount',
   'credit_score'
]

df[numerical_cols] = maxabs.fit_transform(
   df[numerical_cols]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 79

Code:

```
for col in numerical_cols:

   plt.figure(figsize=(7,4))

   sns.histplot(
      df[col],
      kde=True
   )

   plt.title(f'MaxAbs Scaled {col}')

   plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 80

Code:

```
from sklearn.preprocessing import RobustScaler

robust = RobustScaler()

numerical_cols = [
   'annual_income',
   'loan_amount',
   'credit_score'
]

df[numerical_cols] = robust.fit_transform(
   df[numerical_cols]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 81

Code:

```
for col in numerical_cols:

   plt.figure(figsize=(7,4))

   sns.histplot(
      df[col],
      kde=True
   )

   plt.title(f'Robust Scaled {col}')

   plt.show()
```

Interpretation: Visualizes the distribution of numerical variables.

Step 82

Code:

```
from sklearn.preprocessing import Normalizer

normalizer = Normalizer()

numerical_cols = [
   'annual_income',
   'loan_amount',
   'credit_score'
]

df[numerical_cols] = normalizer.fit_transform(
   df[numerical_cols]
)

df.head()
```

Interpretation: Displays the first records to understand dataset structure.

Step 83

Code:

```
df.to_csv(
   'customer_credit_risk_final_dataset.csv',
   index=False
)

print("Final Dataset Exported Successfully!")
```

Interpretation: Performs a preprocessing or analytical operation.

Missing Value Techniques

Mean Imputation: Replaces missing values with the average.
Median Imputation: Uses the middle value.
Mode Imputation: Uses the most frequent category.
KNN Imputation: Predicts missing values from nearest neighbors.

EDA

EDA includes dataset structure analysis, missing value analysis, descriptive statistics, correlation analysis, and visualization of distributions and categories.

Outlier Treatment

Outliers were identified using Z-Score and IQR methods and treated using capping and Winsorization.

Feature Engineering

Additional features such as debt-to-income ratios and date-based variables were created to improve predictive power.

Encoding, Scaling and Transformations

Ordinal Encoding, One-Hot Encoding, Log Transformation, Box-Cox, Yeo-Johnson, StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler and Normalizer were applied.

Steps :

A. Data Understanding

B. Data Quality Report

C. Missing Value Handling

├── Mean Imputation

├─── Median Imputation

├─── Mode Imputation

└─── KNN Imputation

D. Outlier Treatment

├─── Z-Score

├─── IQR

├─── Percentile

└─── Winsorization

E. Feature Engineering

└─── Date Feature Extraction

F. Encoding

├─── Ordinal Encoding

├─── Label Encoding

└─── One-Hot Encoding

G. Binning

├─── Equal Width

├─── Quantile

└─── K-Means

H. Transformation, Scaling & Export

├─── Log

├──── Square Root

├──── Reciprocal

├──── Box-Cox

├──── Yeo-Johnson

├──── StandardScaler

├──── MinMaxScaler

├──── MaxAbsScaler

├──── RobustScaler

├──── Normalizer

└──── Final Dataset Export

Conclusion

The project successfully built a comprehensive preprocessing pipeline suitable for credit risk analysis and predictive modeling.

This project successfully prepared a customer credit risk dataset through comprehensive preprocessing techniques. Missing values were handled, outliers were treated, features were engineered, categorical variables were encoded, transformations were applied, and scaling techniques were performed. The final dataset is clean, consistent, and suitable for predictive analytics and machine learning-based credit risk assessment.